

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: В. Г. Кабанова
Группа: М8О-304БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель лабораторной работы: Познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научитесь работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

Вариант: №9 – Конус

Дополнительные задания(на «4»):

1. Обеспечьте переключение матрицы проекции (ортографическая и перспективная) в интерфейсе
2. Добавить элементы интерфейса для изменения позиции, поворота и растяжения фигуры
3. Добавить возможность движения и поворота фигуры по сложной траектории, сделать элемент интерфейса для паузы и воспроизведения “анимации”, а также возможность изменять скорость “анимации” и другие параметры траектории (например радиус траектории и др.)
4. Добавить возможность изменять цвет фигуры с помощью элементов интерфейса (в ImGui есть ColorEdit для этого)
5. Назначить процедурно цвета для каждой вершины, например в зависимости от позиции вершин в локальной системе координат фигуры, тогда возможность изменять цвет с помощью элементов интерфейса будет умножать выбранный цвет на цвет вершин.

Метод решения

Конус представлен треугольной сеткой. Радиус основания $R = 0,5$, высота $H = 1$. Окружность основания разбивается на 50 сегментов.

Оси: X направлен вправо, Y – вниз, Z – в экран. Верхушка конуса расположена в точке $(0, -H, 0)$, центр основания — в точке $(0, 0, 0)$.

Координаты точек окружности вычисляются по формулам:

$$\varphi_i = \frac{2\pi i}{N}, \quad x_i = R \cos \varphi_i, \quad y_i = 0, \quad z_i = R \sin \varphi_i$$

В функции createConeGeometry() используются следующие вычисления:

```
const float angle = 2.0f * PI * float(i) / float(CONE_SEGMENTS);  
const float x = CONE_RADIUS * cosf(angle);  
const float y = 0.0f;
```

```
const float z = CONE_RADIUS * sinf(angle);
```

Каждая вершина содержит положение и цвет:

```
struct Vertex {
```

```

float position[3];
float color[3];

};

```

Боковая поверхность получается из треугольников, соединяющих верхушку конуса с двумя соседними точками окружности. Основание из треугольников, соединяющих центр основания с соседними точками окружности.

Для соединения последнего сегмента с первым использую: $(i + 1) \% \text{CONE_SEGMENTS}$.

Итоговая модель содержит 52 вершины, 100 треугольников и 300 индексов.

Для хранения геометрии создаются два объекта `VkBuffer`:

- вершинный буфер с флагом `VK_BUFFER_USAGE_VERTEX_BUFFER_BIT`;
- индексный буфер с флагом `VK_BUFFER_USAGE_INDEX_BUFFER_BIT`.

Память выделяется с помощью библиотеки `Vulkan Memory Allocator`, через функцию `vmaCreateBuffer()`. Координаты, цвета и индексы копируются в выделенную память с помощью `memcpy()`.

В функции `render()` буферы привязываются командами `vkCmdBindVertexBuffers()` и `vkCmdBindIndexBuffer()`. Отрисовка выполняется благодаря: `vkCmdDrawIndexed(fd.command_buffer, vk_index_count, 1, 0, 0, 0);`

Задание 1.

Я выполнила перспективную и ортографическую проекции.

Переключение происходит через элементы `ImGui`:

```

ImGui::RadioButton("Perspective", &proj_mode, 0);
ImGui::RadioButton("Orthographic", &proj_mode, 1);

```

Перспективная проекция учитывает расстояние до камеры: при удалении объекта его видимый размер уменьшается. Используется матрица:

$$\begin{pmatrix} \frac{1}{a \tan \frac{\theta}{2}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan \frac{\theta}{2}} & 0 & 0 \\ 0 & 0 & \frac{f}{f-n} & -\frac{fn}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

где θ - вертикальный угол обзора, a - отношение ширины окна к высоте, n и f - расстояния до ближней и дальней плоскостей видимости.

В коде матрица:

```
void mat4_perspective(float fov_y, float aspect, float znear, float zfar, float m[4][4])
{
    memset(m, 0, sizeof(float) * 16);
    const float tan_half = tanf(fov_y * 0.5f);
    m[0][0] = 1.0f / (aspect * tan_half);
    m[1][1] = 1.0f / tan_half;
    m[2][2] = zfar / (zfar - znear);
    m[2][3] = 1.0f;
    m[3][2] = -(zfar * znear) / (zfar - znear);
}
```

Матрицы хранятся по столбцам: первый индекс - столбец, второй - строку.

При ортографической проекции видимые размеры объекта не зависят от расстояния до камеры. Матрица вида:

$$\begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{b-t} & 0 & -\frac{b+t}{b-t} \\ 0 & 0 & \frac{1}{f-n} & -\frac{n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

где l, r, t, b - левая, правая, верхняя и нижняя границы области видимости.

В коде:

```
void mat4_ortho(float left, float right, float bottom, float top, float znear, float zfar, float m[4][4])
{
    memset(m, 0, sizeof(float) * 16);
    m[0][0] = 2.0f / (right - left);
    m[1][1] = 2.0f / (bottom - top);
    m[2][2] = 1.0f / (zfar - znear);
    m[3][0] = -(right + left) / (right - left);
    m[3][1] = -(top + bottom) / (bottom - top);
    m[3][2] = -znear / (zfar - znear);
    m[3][3] = 1.0f;
}
```

Матрицы передаются в шейдер через uniform-буфер VkBuffer. Доступ к нему обеспечивается набором дескрипторов VkDescriptorSet, привязанным к set = 0, binding = 0.

Задание 2.

Для преобразования модели используются однородные координаты и матрицы размером 4 на 4.

Матрица переноса:

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Матрица масштабирования:

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Для вращения используются функции `mat4_rotation_x()`, `mat4_rotation_y()` и `mat4_rotation_z()`. Например, матрица поворота вокруг X:

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Углы из интерфейса переводятся из градусов в радианы.

В коде:

```
mat4_rotation_x(g_rotation_degrees[0] * PI / 180.0f, Rx);  
mat4_rotation_y(g_rotation_degrees[1] * PI / 180.0f + anim_rot_y, Ry);  
mat4_rotation_z(g_rotation_degrees[2] * PI / 180.0f, Rz);
```

Итоговая модельная матрица:

$$M = T \cdot R_y \cdot R_x \cdot R_z \cdot S$$

Задание 3.

Конус движется по окружности в плоскости XZ и одновременно вращается вокруг Y.

Для расчёта анимации определяется время между кадрами:

$$\Delta t = t_{\text{тек}} - t_{\text{пред}}$$

В коде:

```
const double dt = (g_last_time == 0.0) ? 0.0 : (time - g_last_time);  
g_last_time = time;
```

Параметр анимации изменяется с учётом выбранной скорости:

$$\tau_{\text{н}} = \tau_{\text{ст}} + \vartheta \Delta t.$$

```
if (g_anim_playing) {  
    g_anim_time += float(dt) * g_anim_speed;  
}
```

При включённой анимации положение объекта определяется формулами:

$$x = x_0 + r \cos \tau, \quad y = y_0 + h, \quad z = z_0 + r \sin \tau.$$

Дополнительный угол вращения вокруг Y вычисляется как:

$$\beta_{\text{аним}} = \tau\omega$$

где ω - параметр скорости собственного вращения.

Положение и угол включаются в модельную матрицу, которая передаётся через существующий uniform-буфер.

В интерфейсе доступны включение и отключение анимации, изменение скорости, радиуса траектории, вертикального смещения, скорости собственного вращения и сброс времени.

Задание 4.

Для выбора цвета используется элемент ImGui:

```
ImGui::ColorEdit3("Tint", g_tint);
```

Выбранные значения хранятся в массиве `g_tint` и передаются в поле `tint` uniform-буфера:

```
memcpy(vk_uniform_buffer_memory->tint, g_tint, sizeof(g_tint));
```

В шейдере выполняется покомпонентное умножение цвета вершины на выбранный цвет, то есть:

$$R = R_g R_t, \quad G = G_g G_t, \quad B = B_g B_t$$

Белый множитель $(1, 1, 1)$ сохраняет исходную окраску.

Для передачи множителя используется тот же `VkDescriptorSet`, что и для матриц.

Задание 5.

Цвет каждой вершины определяется её положением в локальной системе координат модели.

Координаты X и Z находятся в диапазоне $[-R, R]$. Для преобразования этого диапазона в цветовые значения $[0, 1]$ использую формулы:

$$C_r = \frac{x}{2R} + \frac{1}{2}, \quad C_b = \frac{z}{2R} + \frac{1}{2}$$

Зелёная компонента соответствует:

$$C_g = -\frac{y}{H}.$$

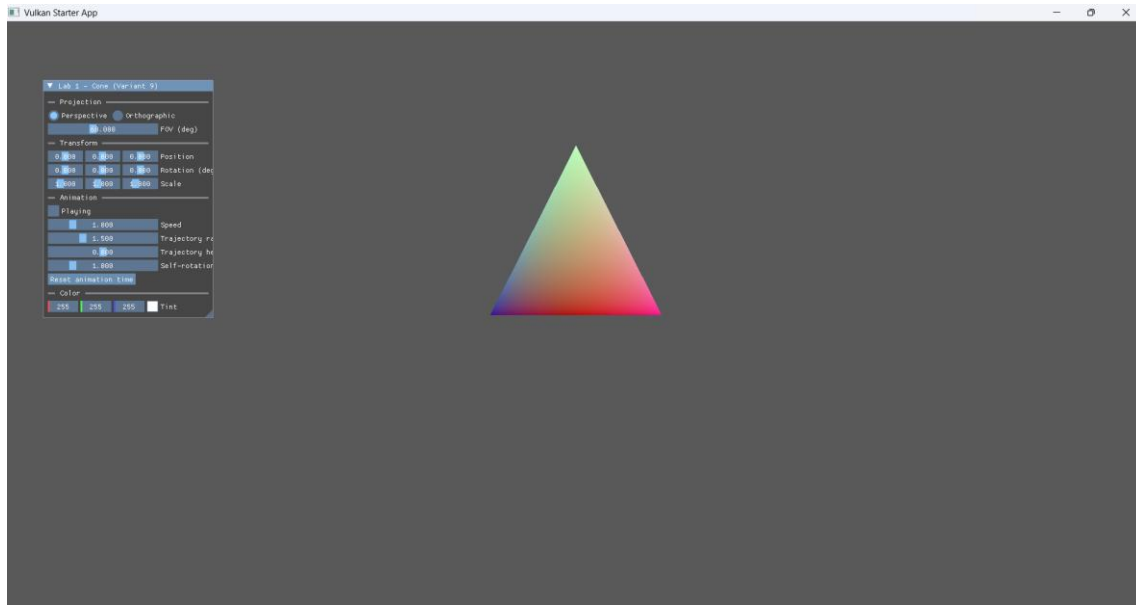
У основания $y = 0$, поэтому $C_g = 0$. У верхушки $y = -H$, поэтому $C_g = 1$.

Цвет хранится в вершинном буфере вместе с координатами. Его описание в графическом конвейере задаётся через `VkVertexInputAttributeDescription`:

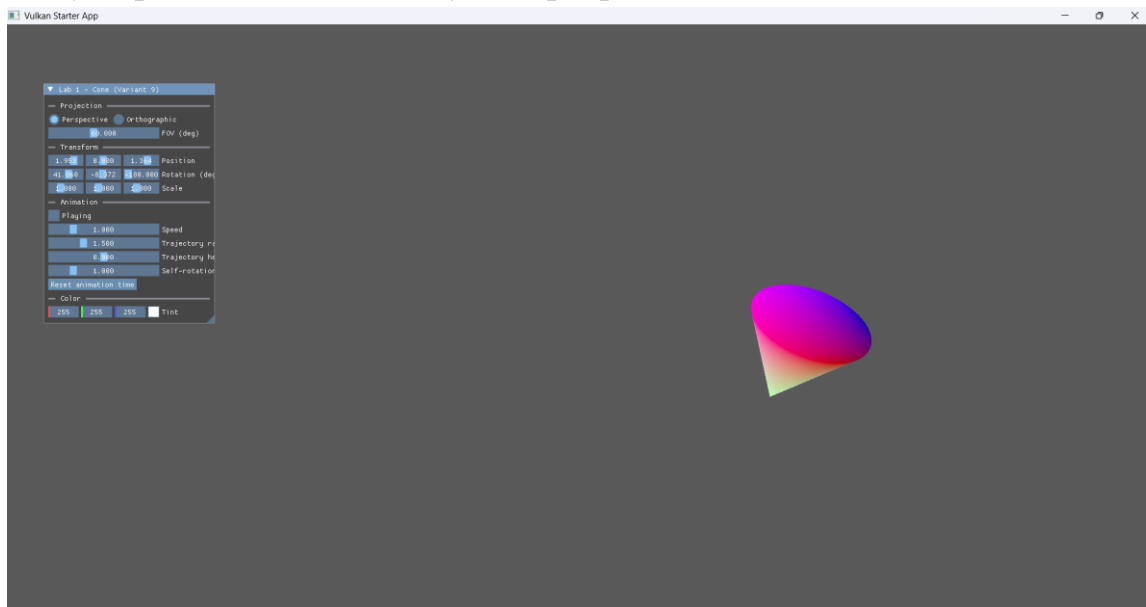
```
.location = 1,
.binding = 0,
.format = VK_FORMAT_R32G32B32_SFLOAT,

.offset = offsetof(Vertex, color),
```

Результаты



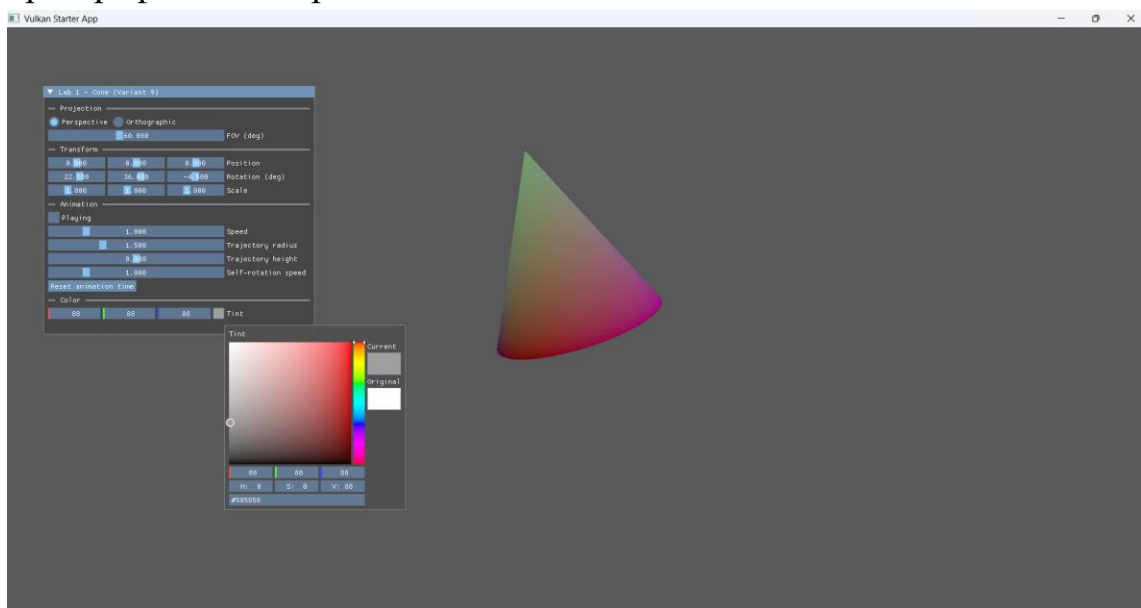
Конус при изначальном запуске программы.



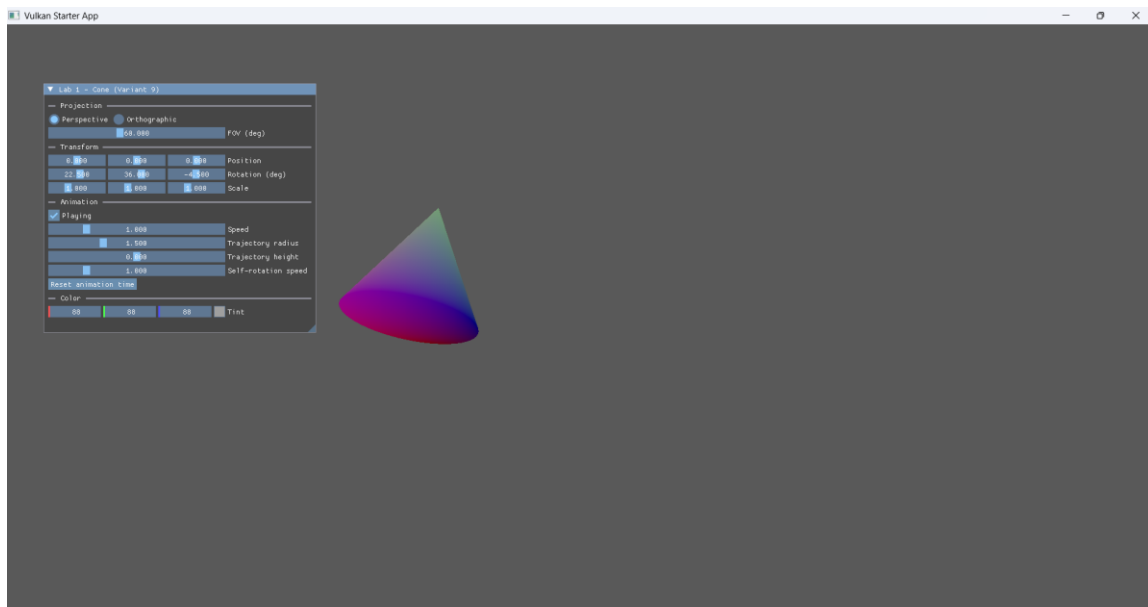
Демонстрация изменения положения и угла поворота при перспективной проекции.



Демонстрация изменения положения и угла поворота при ортографической проекции.



Демонстрация изменения цвета конуса.



Демонстрация изменения положения конуса при движении по траектории.

Вывод

Я научилась строить трёхмерную модель конуса. Освоила создание буферов Vulkan, передачу данных через uniform-буфер и дескрипторы, настройку графического конвейера и запись команд отрисовки. Изучила применение матриц переноса, вращения и масштабирования, а также различия между перспективной и ортографической проекциями. С помощью ImGui я реализовала управление параметрами модели, проекцией, анимацией и цветом. Процедурная окраска позволила связать цвет вершин с их локальными координатами и получить плавные цветовые переходы на поверхности. Реализовала движение по окружности и собственное вращение конуса.